



Shop Floor XML API Specification Early Adopter Guide

Author	MQM Replication Team
Creation Date	24-August 2015
Last Updated	27-October 2015
Version	Draft 1.5



Document Control

Change Record

Date	Author	Version	Change Reference
24-Aug-15	xxx	Draft 1.0	No Previous Document
24-Aug-15	xxx	Draft 1.1	Updated based on review with MQM Replication Team (Eric, Melinda, Max, Joe, Tom)
30-Aug-15	xxx	Draft 1.2	Updated based on review with MQM Replication Team (Eric, Melinda, Max, Joe, Tom)
13-Sep-15	xxx	Draft 1.3	Incorporated comments from Melinda's edit of the document. Incorporated updates based review with MQM Replication Team on Sept 6.
22-Sep-15	Shilpa Kaul	Draft 1.4	1. Heading style and table headers have been changed. 2. Cover page has new look. 3. Changed the font to Myriad Set. 4. Updated - Response XML message will no longer contain the original Payload. It will only contain the UUID and the Errors/Warnings. 5. Updated content - no longer be using an XML namespace 6. Added list of error codes.
20-Oct-15	Shilpa Kaul	Draft 1.5	1. Added 'Zip' details to the tables. 2. Added content to the 'Command Names' and 'Data Payloads' sections. 3. Added a column 'Required' to the tables (where required). 4. Added a section 'Data Compression & Decompression'. 5. Added a sample Compression/Decompression code. 6. Replaced 'Modpartnum' with 'ModAPN'. 7. Added appendix. 8. Added Base64 Encoding/Decoding code

Reviewers

Name	Position
xxx	Apple
xxx	Apple
xxx	Apple
xxx	Oracle

Distribution

Version	Name	Location
All	Library Master	Project Library



Table of Contents

OVERVIEW	4
Introduction	4
Purpose.....	4
Scope.....	4
CM/Apple Responsibilities	5
SHOP FLOOR XML API.....	7
Overview.....	7
Assumptions	7
Request Structure.....	7
SI_Header	7
Command Names.....	9
Data Payloads	9
VALIDATION.....	16
UNIQUE ID GENERATION.....	17
Batch ID	17
Payload UUID.....	17
DATA COMPRESSION & DECOMPRESSION.....	18
Sample Compression/Decompression Code.....	18
Sample Compressed/Base64-Encoded Messages.....	21
ERROR-HANDLING.....	23
COMMUNICATION PROTOCOL.....	26
TCP	26
Message Size.....	26
Synchronous Communication	26
Connection Timeout	26
TCP Connection Re-Use.....	26
Parallel Processing	26
OPEN AND CLOSED ISSUES.....	27
Open Issues	27
Closed Issues.....	27



Overview

Introduction

This document provides a preview of the Shop Floor XML API Specification planned for release at the end of 2011. The API describes a new process that will require Contract Manufacturers (CMs) to send XML messages over a TCP connection and handle XML response messages. This new process is intended to replace the current data interchange infrastructure, which is based around the notion that CMs send data to Apple via flat files. The new API includes a format definition for XML requests and responses, basic data validation rules, and technical and procedural guidelines for submitting data, digesting responses, and re-submitting rejected requests.

The objective of the API is to standardize message formats exchanged between all components of the MQM system (Mac and Indigo), including all CMs. This will help facilitate data quality control and enforcement.

Purpose

This document is intended as an Early Adopter Guide to provide a CM with advanced notification of upcoming changes, so that the CM may begin appropriate planning. This document includes a draft of the planned XML Request and Response formats as well as guidelines regarding data validation, data interchange over TCP, batching payloads into a single XML, and re-processing failed requests.

This document is not intended as the final or complete design of the Shop Floor XML API Specification.

Scope

This Early Adopter Guide will focus on the use-cases for Indigo Module Data only. This means that it will account for the following feed types:

1. Feed Type 2: Indigo FATP Module Data
2. Feed Type 4: Indigo SMT Module Data

However, it should be noted that the Shop Floor XML API will eventually support all of the following feed types:

- Mac and Indigo Module Data, including:
 - Module Data
 - Module Tracking Data
- Mac and Indigo Yield Data, including:
 - Amtran Board Data
 - Board Data



- Manual Fail Data
- Manual Pass Data
- MPU Data
- Power Supply Data
- Yield Data
- Packout Data
- Product Attribute Data
- Serial Link Data

CM/Apple Responsibilities

The XML API will impose the following obligations on the involved parties:

CM Responsibilities

1. Generate XML request messages, compliant with the guidelines herein, including:
 - a. Proper format.
 - b. Valid content.
 - c. Batching of payloads into a single XML request.
 - d. Unique identifier for each XML request.
 - e. Unique identifier for each payload within the XML request.
2. Exchange XML messages over TCP.
 - a. Sending the request.
 - b. Receiving the response.
3. Re-process failed payloads.
 - a. Identifying failed payloads within an XML response.
 - b. Correcting the errors in the rejected data.
 - c. Re-submitting corrected data with the original payload uuid and the reprocessId provided in the response.

Apple Responsibilities

1. Receive XML requests on a TCP port.
2. Validate the request structure.
3. Validate the request data.
4. Process the payloads within the request.
5. Generate an XML response with a response payload for each payload in the original request.
6. Generate a reprocessId for any payloads that are being rejected for containing errors.
7. Check for reprocessId in all incoming request and process resubmits appropriately.



8. Monitor CM compliance with the specified guidelines, including error statistics and re-processing attempts.



Shop Floor XML API

Overview

The Shop Floor XML API defines the XML structure for request and response messages. CMs must generate messages conforming to the Shop Floor XML API and be able to process responses.

Request messages are denoted by the <SI_Message> </SI_Message> tags. Response messages are denoted by <SI_Response> </SI_Response> tags. All messages contain a header component and a data component. The header provides metadata about the data contained within the message (e.g. the payload type). The data component includes one or more payload elements. The number of payload elements in a response XML message matches the number of payload elements in the corresponding request XML message.

All payloads within a single request must be of the same type (e.g. all Module Data payloads).

Assumptions

1. All timestamps are local relative to the timezone of the system on which they are generated. This is true for requests as well as responses.
2. All timestamps use a 24-hour format (e.g. 2011-09-09 15:25:53)
3. All elements are required except where noted otherwise.

Request Structure

The root element of a request is SI_Message. The SI_Message element consists of header (SI_Header) and data (SI_Data) elements, as follows:

SI_Header

The SI_Header is composed of the following elements:

Element Name	Element Description	Sample Value	Required/Optional	Comments
Command_Name	Indicates the feed type corresponding to the payloads contained within the message.	SendModuleData	R	For this Early Adopter effort, the value should always be SendModuleData. Eventually, more values will be supported.
Response_Mode	Indicates whether or not the client is requesting a response.	ACTIVE	R	ACTIVE indicates that the client (CM) expects a response. (PASSIVE indicates that the client does not expect a response. For Shop Floor data, this value should always be ACTIVE).
Site	The site code of the CM sending the request.	FXGL	R	The site code should be pre-defined by the CM and Apple and is specific to each site.



Host	Identification of the CM-owned server sending the request.	12.345.67.890	R	IP Address of the server generating the request
Project_Code	The project code. All payloads in the message should correspond to the same project code.	K48	R	This is the same as the Product Code and Family Type Code.
Commodity_Type	Indicates the type of commodity to which the data corresponds	2	R	For this Early Adopter effort, the value should always be 2 or 4 (2=Indigo FATP Module Data; 4=Indigo SMT/MLB Module Data). Eventually, more values will be supported.
Batch_ID	A unique id for the XML message (i.e. for the batch of payloads in the message). Recommendation is to include the Start_TS, End_TS, Commodity_Type, Project_Code, and an incremental value, using only alphanumeric characters and underscores (see sample value)	20110701050000_20110701130000_2_K48_1000	R	The batch id should be unique per XML message (i.e. per batch of payloads) for the site.
Zip	If included and set to true, indicates that the contents of <SI_Data> are compressed. The first four bytes of the compressed string should indicate the length of the uncompressed byte array.	false	O	If set to true, the contents of SI_Response_Data will also be compressed (with the first four bytes indicating the length of the uncompressed byte array)
Start_TS	The start timestamp of the time window covered by the message.	2011-07-01 05:00:00	R	Format: YYYY-MM-DD HH:MI:SS (24-hour format)
End_TS	The end timestamp of the time window covered by the message.	2011-07-01 14:00:00	R	Format: YYYY-MM-DD HH:MI:SS (24-hour format)
Submit_TS	The time the message was generated at the CM.	2011-07-01 14:05:32	R	Format: YYYY-MM-DD HH:MI:SS (24-hour format)

SI_Data

The SI_Data element contains the payloads to be delivered to an application. Each payload is contained within <Payload> </Payload> tags. Within each set of payload tags are elements specific to the type of data contained within the message, as specified by the Command_Name element in the header.



For the purposes of this Early Adopter Guide, only Payload elements corresponding to SendModuleData will be discussed.

All payloads must be of the same type. Any attempt to include more than one payload type within the same request message will result in a validation failure. Multiple occurrences of a given type are supported (and, in fact, encouraged).

Sample Request XML

```
<?xml version="1.0" encoding="UTF-8"?>
```




```

<SI_Message>
  <SI_Header version="1.0">
    <Command_Name>SendModuleData</Command_Name>
    <Response_Mode>ACTIVE</Response_Mode>
    <Site>FXGL</Site>
    <Host>12.345.67.890</Host>
    <Project_Code>K48</Project_Coda>
    <Commodity_Type>2</Commodity_Type>
    <Batch_ID>20110701050000_20110701130000_2_K48_1000</Batch_ID>
    <Zip> False</Zip>
    <Start_TS>2011-07-01 05:00:00</Start_TS>
    <End_TS>2011-07-01 14:00:00</End_TS>
    <Submit_TS>2011-07-01 14:05:32</Submit_TS>
  </SI_Header>
  <SI_Data>
    + <Payload>
    + <Payload>
    + <Payload>
  </SI_Data>
</SI_Message>

```



+ Indicates the presence of child elements.

Command Names

The following table shows the mapping of shop floor feed types to command names. Note that multiple feed types may map to the same command name:

Command Name	New Feed Type	New Commodity Type	Mac/Indigo/Other	Old Feed Type	Comments
SendModuleData	MOD	FATP	All	2, MOD-TRACK	Mod Track data for all finished goods.
		SMT	All	4, BOARD-MOD-TRACK, MOD-TRACK	Mod Track data for all sub-assemblies

Data Payloads

The following sections will describe the different payload types valid for <SI_Data> including required field formats, required/optional, allowable values and other supporting details. For the Early Adopter Guide, only ModuleData will be covered.

SendModuleData Payload

This payload structure corresponds to the command name "SendModuleData."



Supported Legacy Feeds

All of the following older formats should now use this new format:

- Module Data (Indigo FATP)
- Module Data (Indigo SMT)
- Module Tracking Data (Mac FATP)
- Module Tracking Data (Mac SMT)
- Board Data (Mac)

Elements

The Payload element is composed of the following elements:

Element Name	Element Description	Sample Value	Comments
ReprocessID	Indicates this record is being reprocessed. The value must be taken from the response that reported errors for this data.	5jfywdb-d906-4432-rt2b-5104nhg37d90	Required IF the data is being reprocessed. The value must be the same as the reprocessId provided by Apple in the response. If the payload is being submitted for the first time, this field should not be included.
UUID	A unique id for the payload.	51b4bbdb-d616-4602-b92b-510447250b9b	If a record needs to be re-submitted for any reason, the same uuid must be included with the payload every time it is submitted.
UnitSN	For FATP data, the unit serial number. For SMT data, the MLB or SA serial number.	DN6G8C8CDKNV	Should be the same as unitPlantCode + unitWom + unitSequence + unitConfigCode + unitSuffix
UnitPlantCode	2 or 3-character code that identifies the factory location.	DN6	
UnitWom	Week of manufacturing	G8	
UnitSequence	Sequential number to make unitSN unique.	C8C	
UnitConfigCode	Code identifying the configuration of the unit.	DKNV	
UnitSuffix	Revision Number and Check Digit		Optional Must be included if the unitSN has a suffix.
ModelKey	This could be a Part Number of a finished good, a Purchase Order Number, the Marketing Part Number, CID (vendor identifier), a user-visible Marketing Part Number, or a Sales Order Number.	MA501LL/A	
TS	Timestamp of when the model was scanned	2011-07-01 13:28:31	Format: YYYY-MM-DD HH:MI:SS (24-hour format)
ModAPN	Apple part number	646-0730	
ModDesc	Part description	PWRSPLY	



ModSN	Part serial number	1X1252W7VGSRB	Required if part is serialized. Also required if lotCode and dateCode are not provided.
ModPlantCode	2 or 3-character code that identifies the vendor who made the part	1X	Should correspond to the vendorName.
ModWom	Week of manufacturing of the module	125	
ModSequence	Sequential number to make modSN unique	2W7V	
ModEEEE	3 or 4-character code identifying the configuration of the module	GSR	
ModSuffix	Revision Number and Check Digit	B	Optional Must be included if the modSN has a suffix.
LineName	Line on which the model was scanned	CO3-2FT-01A	
LotCode	Used when the vendor has already coded a batch of materials/parts with a tracking number	DKF7FPC18171921000W0000J	Required if modSN and dateCode are not provided.
DateCode		0814	Required if modSN and lotCode are not provided.
VendorName	Text name of the module/part vendor	Flextronics	Should correspond to the modPlantCode.
ActionFlag	Indicates if the part is being added ("A") or removed ("D")	A	
InfoCode	Additional description for a module that is unique to this product	U1_RF	Optional Should only be used if the information is different from modDesc.

Sample XML Payload for Indigo FATP Module Data:

```

<Payload>
  <ReprocessID>5jfywdb-d906-4432-rt2b-5104nhg37d90</ReprocessID>
  <UUID>51b4bbdb-d616-4602-b92b-510447250b9b</UUID>
  <UnitSN>DN6G8C8CDKNV</UnitSN>
  <UnitPlantCode>DN6</UnitPlantCode>
  <UnitWom>G8</UnitWom>
  <UnitSequence>C8C</UnitSequence>
  <UnitConfigCode>DKNV</UnitConfigCode>
  <ModelKey>MA501LL/A</ModelKey>
  <TS>2011-07-01 13:28:31</TS>
  <ModAPN>646-0730</ModAPN>
  <ModDesc>PWRSPPLY</ModDesc>
  <ModSN>1X1252W7VGSRB</ModSN>
  <ModPlantCode>1X</ModPlantCode>
  <ModWom>125</ModWom>
  <ModSequence>2W7V</ModSequence>
  <ModEEEE>GSR</ModEEEE>
  <ModSuffix>B</ModSuffix>
  <LineName>CO3-2FT-01A</LineName>
  <LotCode>DKF7FPC18171921000W0000J</LotCode>
  <DateCode>0814</DateCode>
  <VendorName>Flextronics</VendorName>
  <ActionFlag>A</ActionFlag>
  <InfoCode>U1_RF</InfoCode>
</Payload>

```



The unit suffix element tag is not included because there was no value. This field is optional.

Response Structure

The root element of a response is SI_Response. The SI_Response element consists of header (SI_Response_Header) and data (SI_Response_Data) elements, as described below.

SI_Response_Header

The SI_Response_Header element is composed of the following elements:

Element Name	Element Description	Sample Value	Required	Comments
Command_Name	Indicates the feed type corresponding to the payload responses contained within the message.	SendModuleData	R	This will be the same as the command name in the corresponding request. Hence, for this Early Adopter effort, the value will be SendModuleData.
Supported	Indicates whether or not the command is supported by the target application	SUPPORTED	R	For any valid command name, this value will always be SUPPORTED. Otherwise, the value will be NOT SUPPORTED.
Responses_Sent	Number of responses sent	1	R	This value will always be 1.
Responses_Expected	Number of responses expected	1	R	This value will always be 1.
Batch_ID	The Batch_ID of the corresponding request	20110701050000_ 20110701130000_ 2_K48_1000	R	
Zip	If included and set to true, indicates the contents of SI_Response_Data are compressed. The first four bytes of the compressed contents will indicate the length of the uncompressed byte array.	false	O	Will be true if and only if the request had this value set to true
App_Name	Name of the application sending the response	mqmmessagemanager-7800	R	The name of the application sending the response.
Sequence_Number	Sequence number of the message	1	R	This value will always be 1.
Error_Code	General error code indicating whether or not any of the payloads encountered an error or warnings	0	R	0 indicates success; any other value indicates an error or warning in at least one of the payloads. See Error-Handling section for a list of possible values.
Error_Msg	Message corresponding to the error code		R	If the Error_Code field was set to a value other than zero, then this field will contain an error message. If the Error_Code field was set to zero then this field is blank. See Error-Handling section for a list of possible values.



Host_Name	Host name of the application sending the response	Server01	O	Optional. The host name of the application sending the response.
IP_Address	IP address of the application sending the response	98.765.43.210	O	Optional. The IP Address of the application sending the response.
TCP_Port	TCP port (listening) of the application sending the message	7800	O	Optional. The TCP port on which the application sending the response is listening for requests.
Response_Timestamp	Timestamp indicating when the response was generated	2011-07-01 14:05:33	R	Format: YYYY-MM-DD HH:MI:SS (24-hour format)

SI_Response_Data

The SI_Response_Data element contains one or more SI_Response_Payload elements (one for every payload in the corresponding request). Each SI_Response_Payload element in turn contains the original payload's uuid element, at least one Errors element, and, if appropriate, repeating Warnings elements.

SI_Response_Payload

The SI_Response_Payload element is composed of the following elements:

Element Name	Element Description	Sample Value	Comments
ReprocessID	Provided only for payloads that are being rejected because they contain errors. Otherwise, this field will not be present in the response.	5jfywdb-d906-4432-rt2b-5104nhg37d90	The CM MUST include this reprocessId when they resubmit the same data, after making the required corrections.
UUID	The uuid of the payload in the request to which this response payload corresponds.	51b4bbdb-d616-4602-b92b-510447250b9b	
Errors	A repeating element containing child elements that specify the error code and error message. There will be an "Errors" element for each error encountered during processing of the request.	N/A (See Error_Code and Error_Msg)	There will be at least 1 Errors element, and possibly more. The CM should evaluate all of these to correct a rejected payload before resubmitting the payload. If a payload was processed successfully, there will be only one Errors element AND the error code will be 0.
Code	This is a child of the Errors element.	0	0 indicates success; any other value indicates an error. See Error-Handling section for a list of possible values.
Msg	This is a child of the Errors element.		Present only if Error_Code is not 0. See Error-Handling section for a list of possible values.
Field	This is a child of the Errors element.		The name of the field that has an error.
Value	This is a child of the Errors element.		The value of the field that has an error.
Warnings	A repeating element containing child elements that specify the error code and error message. There will be a "Warnings" element for each warning encountered during processing of the request.	N/A (See Warning_Code and Warning_Msg)	Optional. This element will only be present if warnings were encountered. Warnings indicate non-fatal problems with the request. See Error-Handling section for a list of possible values.



Code	This is a child of the Warnings element.	0	Optional. This element will only be present if warnings were encountered. 0 indicates success; any other value indicates a warning. See Error-Handling section for a list of possible values.
Msg	This is a child of the Warnings element.		Optional. This element will only be present if warnings were encountered. See Error-Handling section for a list of possible values.
Field	This is a child of the Warnings element.		The name of the field that has a warning.
Value	This is a child of the Warnings element.		The value of the field that has a warning.
Payload	The payload of the request message.		Optional. This will be included only if errors or warnings were encountered with the request payload. The fields in the payload will be exactly what was submitted by the CM.

Sample Response XML

```
<?xml version="1.0" encoding="UTF-8"?>
<SI_Response>
  <SI_Response_Header version="1.0">
    <Command_Name>SendModuleData</Command_Name>
    <Supported>SUPPORTED</Supported>
    <Responses_Sent>1</Responses_Sent>
    <Responses_Expected>1</Responses_Expected>
    <Batch_ID>20110701050000_20110701130000_2_K48_1000 </Batch_ID>
    <Zip>>false</Zip>
    <App_Name>mqmmessagemanager-7800</App_Name>
    <Host_Name>Server01</Host_Name>
    <IP_Address>98.765.43.210</IP_Address>
    <TCP_Port>7800</TCP_Port>
    <Sequence_Number>1</Sequence_Number>
    <Error_Code>1001</Error_Code>
    <Error_Msg>Errors: correct and reprocess rejected payloads</Error_Msg>
    <Response_Timestamp>2011-07-01 14:05:33</Response_Timestamp>
  </SI_Response_Header>
  <SI_Response_Data>
    <SI_Response_Payload>
      <UUID>5134rty7-d797-2346-b92b-105447250ii9</UUID>
      <Errors>
        <Code>0</Code>
      </Errors>
    </SI_Response_Payload>
    <SI_Response_Payload>
      <ReprocessID>5jfywdb-d906-4432-rt2b-5104nhg37d90</ReprocessID>
      <UUID>51b4bbdb-d616-4602-b92b-510447250b9b</UUID>
      <Errors>
        <Code>7</Code>
        <Msg>Data format error in required parameter(s)</Msg>
        <Field>LotCode</Name>
        <Value>DKF7FPC18171921000W0000J 646-0730-XYZ</Value>
      </Errors>
      <Errors>
        <Code>4</Code>
        <Msg>Cannot determine a production line</Msg>
        <Field>LineName</Field>
        <Value>CO3-2FT-01A</Value>
      </Errors>
    </SI_Response_Payload>
  </SI_Response_Data>
</SI_Response>
```



```

<Warnings>
  <Code>7</Code>
  <Msg>Data format error in required parameter(s)</Msg>
  <Field>modPartNum</Field>
  <Value>646-0730-XYZ</Value>
</Warnings>
<Warnings>
  <Code>7</Code>
  <Msg>Data format error in required parameter(s)</Msg>
  <Field>modSN</Field>
  <Value>DQ6B995HT5DMGWA2XN</Value>
</Warnings>
<Payload>
  <UUID>51b4bbdb-d616-4602-b92b-510447250b9b</UUID>
  <UnitSN>DN6G8C8CDKNV</UnitSN>
  <UnitPlantCode>DN6</UnitPlantCode>
  <UnitWom>G8</UnitWom>
  <UnitSequence>C8C</UnitSequence>
  <UnitConfigCode>DKNV</UnitConfigCode>
  <ModelKey>MA501LL/A</ModelKey>
  <TS>2011-07-01 13:28:31</TS>
  <ModAPN>646-0730-XYZ</ModAPN>
  <ModDesc>PWRSPPLY</ModDesc>
  <ModSN>DQ6B995HT5DMGWA2XN</ModSN>
  <ModPlantCode>DQ6</ModPlantCode>
  <ModWom>B995</ModWom>
  <ModSequence>HT5D</ModSequence>
  <ModEEEE>MGWA</ModEEEE>
  <ModSuffix>2XN</ModSuffix>
  <LineName>CO3-2FT-01A</LineName>
  <LotCode>DKF7FPC18171921000W0000J 646-0730-XYZ</LotCode>
  <DateCode>0814</DateCode>
  <VendorName>Flextronics</VendorName>
  <ActionFlag>A</ActionFlag>
  <InfoCode>U1_RF</InfoCode>
</Payload>
</SI_Response_Payload>
</SI_Response_Data>
</SI_Response>

```



+ Indicates the presence of child elements.



Validation

Request messages will be validated against an XML schema to ensure basic conformance to the established XML request structure. This level of validation will check element order, field lengths, data types, patterns, and multiplicity.

Subsequently, request data will also be validated against reference tables in Apple databases.

Further information regarding the validation requirements of each element will be provided at a future time. - *See Open Issue 3.*



Unique ID Generation

The CM is responsible for generating unique ids for each payload in a request (the UUID of each Payload), as well as a unique id for the whole request (the Batch_ID of the request).

Batch ID

The Batch_ID uniquely identifies an XML request message. The Batch_ID must be generated by the CM.

Include notes on the format (max length, character restrictions, etc) - *See Open Issue 3*

Payload UUID

The payload UUID uniquely identifies an individual payload within the request message. The payload UUID must be generated by the CM. The CM should be able to correlate the payload uuid with the data in their source system that was used to generate this payload. In other words, if the CM needs to submit the same payload twice (e.g. after making corrections), they must be able to send the same uuid with the payload.

The payload UUID is independent of the Batch ID, meaning that a payload that is submitted twice (in case that the first attempt failed, for example) must use the same UUID, but it can be sent in a batch with a different Batch ID.

It is the obligation of the CM to ensure that each payload has a unique identifier (the UUID) and that the same UUID is always included when that payload is being sent in a request.

Include notes on the format (max length, character restrictions, etc) - *See Open Issue 3*



Data Compression & Decompression

In order to utilize network bandwidth more efficiently, the CM can choose to compress the contents of the SI_Data node in a request XML message (i.e. compressed Payloads). If a request message contains compressed Payloads, the Zip element in the header must have a value of true. Otherwise, the Zip element can be left out of the header (or set to false).



Only the contents of SI_Data (for a request) or SI_Response_Data (for a response) - not including the tags themselves - can be compressed, not the entire XML message. Also, all of the data must be compressed as a single string. The first 4 bytes of the compressed string must indicate the length of the uncompressed byte array (i.e. length of uncompressed data + 4).

Sample Java code for compressing and decompressing request and response data is included below.

Sample Compression/Decompression Code

This code is intended only as a guideline to show how the request and response payloads should be compressed and encoded (decoded/decompressed). References to methods and variables not included in the samples below are considered unnecessary for the purposes of this example.

Base64 Encoding/Decoding

```

package com.apple.mqm.util;
import java.util.Iterator;
import java.util.List;
import org.apache.commons.codec.binary.Base64;
import org.dom4j.Document;
import org.dom4j.Element;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Codec {
    private static final Logger logger = LoggerFactory.getLogger(Codec.class);
    private final static int REQUEST = 0;
    private final static int RESPONSE = 1;

    public static String getCompressedRequest(String source) throws MMException {
        return compressPayload(source, REQUEST);
    }

    public static String getDecompressedRequest(String source) throws MMException {
        return decompressPayload(source, REQUEST);
    }

    public static String getCompressedResponse(String source) throws MMException {
        return compressPayload(source, RESPONSE);
    }

    public static String getDecompressedResponse(String source) throws MMException {
        return decompressPayload(source, RESPONSE);
    }

    private static String compressPayload(String source, int type) throws MMException {
        logger.info("Source Data Size: {}", source.length());
        Document document = XMLUtils.getDocument(source);
        Element data = (Element)document.selectSingleNode(getDataXPath(type));
        List payloads = document.selectNodes(getPayloadXPath(type));

```



```

        Iterator itr = payloads.iterator();
        StringBuilder builder = new StringBuilder(payloads.size() * 512);
        while(itr.hasNext()){
            builder.append( ((Element)itr.next()).asXML());
        }
        byte[] compressedByteArray = CompressionUtility.compress(builder.toString());
        String compressedData = new String(Base64.encodeBase64(compressedByteArray));
        data.clearContent();
        data.addText(compressedData);
        logger.info("Compressed Data size: {}", document.asXML().length());
        return document.asXML();
    }

    private static String decompressPayload(String source, int type) throws MMException {
        Document document = XMLUtils.getDocument(source);
        Element data = (Element)document.selectSingleNode(getDataXPath(type));
        System.out.println(data.getText());
        String inflatedPayloads =
            CompressionUtility.decompress(Base64.decodeBase64(data.getText().getBytes()));
        logger.info("Inflated Payload: {}", inflatedPayloads);
        data.clearContent();
        data.setContent(getInflatedPayloadsAsList(inflatedPayloads, type));
        return document.asXML();
    }

    private static List getInflatedPayloadsAsList(String inflatedPayloads, int type) throws MMException {
        StringBuilder builder = new StringBuilder(512);
        if(REQUEST == type){
            builder.append(MMConstants.XML_REQUEST_DATA_START_TAG);
            builder.append(inflatedPayloads);
            builder.append(MMConstants.XML_REQUEST_DATA_END_TAG);
        }else{
            builder.append(MMConstants.XML_RESPONSE_DATA_START_TAG);
            builder.append(inflatedPayloads);
            builder.append(MMConstants.XML_RESPONSE_DATA_END_TAG);
        }
        return XMLUtils.getDocument(builder.toString()).getRootElement().elements();
    }

    private static String getDataXPath(int type){
        if(REQUEST == type){
            return MMConstants.XPATH_REQUEST_DATA;
        }
        return MMConstants.XPATH_RESPONSE_DATA;
    }

    private static String getPayloadXPath(int type){
        if(REQUEST == type){
            return MMConstants.XPATH_REQUEST_PAYLOAD;
        }
        return MMConstants.XPATH_RESPONSE_PAYLOAD;
    }
}

```

Data Compression/Decompression

```

package com.apple.mqm.util;
import java.io.UnsupportedEncodingException;
import java.util.zip.Deflater;
import java.util.zip.Inflater;
import org.slf4j.Logger;

```



```
import org.slf4j.LoggerFactory;

public class CompressionUtility {
    private static final Logger logger = LoggerFactory
        .getLogger(CompressionUtility.class);
    // private static final int BUFFER_SIZE = 1024;
    public static byte[] compress(String inputString) {
        byte[] output = null;
        byte[] finaloutput = null;
        try {
            byte[] input = inputString.getBytes("UTF-8");
            // Compress the bytes; store the compressed data into a byte[] of
            // size BUFFER_SIZE
            Deflater compressor = new Deflater();
            output = new byte[inputString.length()];
            compressor.setInput(input);
            compressor.finish();
            int compressedDataLength = compressor.deflate(output);
            // Create a new byte[] of ( size 4 + compressed data size)
            finaloutput = new byte[compressedDataLength + 4];
            // store the length of compressed data in first four bytes
            System.arraycopy(inputToByteArray(input.length), 0, finaloutput, 0, 4);
            // store the compressed byte[] in the remaining bytes
            System.arraycopy(output, 0, finaloutput, 4, compressedDataLength);
        } catch (UnsupportedEncodingException une) {
            logger.error(une.toString());
        }
        return finaloutput;
    }

    public static String decompress(byte[] data) throws MMException {
        Inflater decompressor = new Inflater();
        String outputString = null;
        try {
            int uncompressedSize = byteArrayToInt(data);
            decompressor.setInput(data, 4, data.length - 4);
            byte[] result = new byte[uncompressedSize];
            int resultLength = decompressor.inflate(result);
            decompressor.end();
            // Decode the bytes into a String
            outputString = new String(result, 0, resultLength, "UTF-8");
        } catch (Exception e) {
            if (data == null) {
                logger.error("Error decompressing byte[] [value=null]", e);
                throw new MMException(MMErrors.REQUEST_DECOMPRESSION_ERROR,
```



```

        ErrorUtils.getExceptionDetails(e) + " (byte[] length=0)");
    } else {
        logger.error("Error decompressing byte[] [value={}]",
            data.toString(), e);
        throw new MMException(MMErrors.REQUEST_DECOMPRESSION_ERROR,
            ErrorUtils.getExceptionDetails(e) + " (byte[] length="
                + data.length + ")");
    }
}
return outputString;
}
private static final byte[] intToByteArray(int value) {
    return new byte[] { (byte) (value >>> 24), (byte) (value >>> 16),
        (byte) (value >>> 8), (byte) value };
}
private static final int byteArrayToInt(byte[] b) {
    return (b[0] << 24) + ((b[1] & 0xFF) << 16) + ((b[2] & 0xFF) << 8)
        + (b[3] & 0xFF);
}
}

```

Sample Compressed/Base64-Encoded Messages

Sample Request Message

```

SI_Response_Header version="1.0">
  <Command_Name>SendModuleData</Command_Name>
  <Supported>SUPPORTED</Supported>

<?xml version="1.0" encoding="utf-8"?>
<SI_Message>
<SI_Header>
  <Command_Name>SendModuleData</Command_Name>
  <Response_Mode>ACTIVE</Response_Mode>
  <Site>CSMC</Site>
  <Host>17.104.27.232</Host>
  <Project_Code>N81</Project_Code>
  <Commodity_Type>2</Commodity_Type>
  <Batch_ID>CSMC_2_20111018221630_1-3</Batch_ID>
  <Zip>true</Zip>
  <Start_TS>2011-08-01 15:58:32</Start_TS>
  <End_TS>2011-08-01 16:10:45</End_TS>
  <Submit_TS>2011-10-20 14:38:05</Submit_TS>
</SI_Header>
  <SI_Data>AAADInicvZJvS8MwEMa/il+gJJf+WVeOQJvMUUxLtZ1/XpVilwy2FrS
+2Lc3yXQMN1QEfxC8zyV3z4/
Dqtttxq7nuFrIkou6EC1rGQUACjFjEPm0BSTOxdWwnuqSC3G9jGhApahmxtqL2NTcvvNo7FG4gC
gBmgQhEqNjMfZV9zyVr1uepU2zuHlAcqRZX+qXR
+IqO4E5336r1oMuu602nhonMfaaqyWSjxplN2lbGT8fnkYnssvGgwjJQUByyPmTwOx8YJDyOLBZ
UG+u9I4XYu7PICKpW3qvfcYRmGc

```



```
+OwER6GysyjsTPAhoOfTLOVdJVyXc94ZZan8CpJN9g0mlZeL30Py/
+squKqBGfoQ3VNQBZ379ATG3x3MGxhW8Zw=</SI_Data>
</SI_Message>
```

Sample Response Message

```
<?xml version="1.0" encoding="utf-8"?>
<SI_Response>
  <SI_Response_Header version="1.0">
    <Command_Name>SendModuleData</Command_Name>
    <Supported>SUPPORTED</Supported>
    <Responses_Sent>1</Responses_Sent>
    <Responses_Expected>1</Responses_Expected>
    <Batch_ID>CSMC_2_20111018221630_1-3</Batch_ID>
    <Zip>true</Zip>
    <App_Name>messagemanager-6800</App_Name>
    <Host_Name>biggas01.corp.apple.com</Host_Name>
    <IP_Address>17.104.27.232</IP_Address>
    <TCP_Port>0000</TCP_Port>
    <Sequence_Number>1</Sequence_Number>
    <Error_Code>1001</Error_Code>
    <Error_Msg>One or more payloads failed</Error_Msg>
    <Response_Timestamp>2011-10-20 14:38:07</Response_Timestamp>
  </SI_Response_Header>
  <SI_Response_Data>AAAlgXic7VnRbtowFP2V/QCynUBKkWUp2KKG5tAsCXR9iiJwKySlt5A
+9O9nm1J1tFqF5HZqYl6w7oVjH58r5RwF53GZif0vWe9FmVaPW1mtCV4sYkZontDSKz2IElJo7Hk
o8GGJMDBdHDWNbPYEz
+KIM6JrGBzWeBnyRQQILm7TiERZdp1hYNaYXrOIBBiYbwyoGCdY9aZNt1XdUrKWNkFv5M4mXC
5+P4h6ZFwIVN3m3ubxBOftf0uHq2d0intlHZK/2+lrV
+mwrR4lwrN9IX2lXSkPrbglqJey2Ze7awdMFy1G1nPttW9o
+xG2+l8gniWpTYPoDmh9MdVAleQ0fRCtQ5FXORE/28AxwOlvaqFggubkOFLb5wTr
+amadv6wl9OwKKLsFoMXNd1nYr8CZqV38Exfw/
JNLfSlqR6XZgAJv8LguMasaoVeqX5c30IT9GbFACmuzwUMnnmCNwPFuSnDcynjyzkS5z2d0k7pb
indS1fSS9Ldt2KOshvtjup8lqX+O2Ugxl6mjOPTnST00r/gHIQmKRxqpxlkNBmNJ773KoMkfPpm/
tB7lh8N4Wg5ZuwmppeZXpvMUTKYh+1cy0czeySY8nkcfmUx8l0y+nltxftUp7ZTultK9dDK9JN19+
+You9HuqM5nWeqPfv9Bel48FXlQ8BminsBLH75KIJ/9auQPy6FTmQ==</SI_Response_Data>
</SI_Response>
```



Error-Handling

Upon receiving a response, the CM must determine whether or not any of the payloads in the message contain errors. The CM can reference the `Error_Code` element in the `SI_Response_Header`. An `Error_Code` of 0 in the header indicates that all payloads were processed successfully.

However, if the `Error_Code` in the `SI_Response_Header` is not 0, the CM must inspect each payload's individual `Errors` element. If the CM identifies a Payload with an error code indicating that the message must be re-submitted, the CM must correct the error(s) in the payload and re-send it. Upon re-sending a payload, the client must make sure to include the same UUID used the first time the payload was submitted.

A response message may also contain Warnings. Warnings are non-fatal problems found in the data. The CM should endeavor to avoid these problems in subsequent requests and can choose to correct and resubmit data with warnings.

Further information on the specific error codes that indicate that a payload must be re-submitted will be provided at a future time. - See *Open Issue 4*.



The following information will be updated when the complete error code list is available.

The error codes that may be found in the response `SI_Response_Header` are:

Header Error Code	Header Error Message	Comments
0		All payloads passed validation without errors or warnings.
1000	Accepted with warnings	All payloads were accepted, but some validation problems were found. CM should review warnings and correct their source system to avoid the same warnings in the future.
1001	Some payloads rejected	At least one payload was rejected. CM must check the error codes in each payload and, if any error code is NOT equal to 0, they must correct the data and resubmit the record. The CM must use the <code>reprocessId</code> provided in the response for the rejected payload when reprocessing the data.

The error codes that may be found in the `SI_Response_Payload` are:

Payload Error/Warning Code	Payload Error/Warning Message	Comments
0		Payload passed validation without errors or warnings.



Payload Error/ Warning Code	Payload Error/Warning Message	Comments
2	Invalid WIP/Serial Number	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
3	Missing WIP/Serial Number	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
4	Cannot determine a production line	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
5	Cannot determine a yield point	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
6	Missing required parameter(s)	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
7	Data format error in required parameter(s)	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
8	Required parameter(s) missing data	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
9	Internal software exception	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.
10	Database exception	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.
14	Wrong commodity type	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
19	Timestamp malformed	CM should correct the data in their source system and resubmit the data. CM must include the reprocessId provided in the response for this payload when resubmitting the data.
29	MM timed out waiting for target to respond	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.
30	MM encountered an error trying to connect to a target	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.
31	All targets are unavailable	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.



Payload Error/ Warning Code	Payload Error/Warning Message	Comments
32	MM encountered an internal exception	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.
33	MM reached connection limit. Refusing new connection attempt	CM should try re-sending the same data again. No data corrections needed. A reprocessId will NOT be provided for payloads with this error code.



Communication Protocol

TCP

The client sending the request is expected to establish a TCP connection to the target application (hosted on an Apple server).

Message Size

The client is expected to try to maximize the number of payloads sent within a particular XML request up to a pre-defined limit of [See Open Issue 1] number of payloads). If adding a payload to an existing XML request causes the request size to exceed the pre-defined limit, it should not be added to the existing XML request but rather sent in the next XML request.

Synchronous Communication

The client should send a request XML message over the TCP socket and wait for a response back on the same socket.

Connection Timeout

The client should wait [See Open Issue 2] seconds before timing out. If a timeout occurs, the client must close the connection, open a new connection, and try sending the same message again.

TCP Connection Re-Use

After receiving the response, the CM should try to re-use the same connection to send subsequent messages.

Parallel Processing

The client is expected to open multiple connections to the target application in order to achieve higher levels of throughput. The server is configured to accept a maximum number of concurrent connections and refuse any additional connections until one of the existing connections is closed. If a client tries to establish a connection while the maximum number of concurrent connections is exhausted, the client will receive a response XML message indicating that the connection was refused. In this case, the client must try re-sending the message.



Open and Closed Issues

Open Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date
1	Determine number of payloads to allow per batch				
2	Determine the timeout that a CM should use before closing a connection and resubmitting a batch.				
3	Provide validation rules, including element order, min/max lengths for all fields, allowed characters, data patterns, case-sensitivity, pre-defined lists of allowed values or values to expect.				
4	Provide complete list of error codes, error messages, warning codes, and warning messages that a CM can expect in both the response header and the response payload.				

Closed Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date